Version-Aware Evidence Contracts for Local Security Alert Explanations

Ananay Srivastava, Jitendra Singh, Manik Bhardwaj, Ashish Jain, Mukunj Singh

Department of Computer Science and Engineering, Sharda University, Greater Noida, India

***Abstract*—A local language model can misrepresent a security alert when authorized evidence is incomplete, contradictory, or later withdrawn. We implement a finite claim contract over versioned recipient-specific snapshots. An independent validator admits only claims matching authorized evidence identifiers, exact versions, scope, and permitted predicates; displayed text is rendered from controlled templates. Snapshot changes make dependent reports stale. One repair attempt is allowed before a deterministic fallback. In an eight-case frozen CPU pilot with Qwen3-4B Q4_K_M, direct generation produced 10 unsupported claims among 20 claims, citation-prompted generation produced eight among 20, and verification without lifecycle checks retained three unsupported claims among 48. The complete method emitted no unsupported claims among 46, but fell back in all eight cases; median generation time was 146.7 seconds. These results show a safety boundary and a severe usability limitation, not a demonstrated benefit from model prose. No analyst study or field accuracy result is claimed.**

***Index Terms*—security alerts, local inference, evidence versions, claim verification, abstention**

I. Introduction

An alert may state that an authentication anomaly occurred while withholding identities, source addresses, or exact counts. A fluent explanation can fill these gaps with plausible details that the alert never established. Running the model locally changes where inference occurs; it does not make the resulting statements faithful to the supplied evidence.

Local cybersecurity models such as Hackphyr and RedSage establish relevant model capabilities [1], [2], while citation evaluation and post-generation revision are already studied in ALCE and RARR [3], [4]. The question here is narrower: how should a security explanation preserve unavailable, disputed, and revoked evidence instead of silently turning it into a complete narrative?

We implement a typed claim contract and a versioned report lifecycle. The contract restricts statements eligible for deterministic rendering, retains unresolved claims, and records dependencies for later invalidation. It does not make a language model an authoritative detector. A small frozen pilot tests claim support and latency; no analyst study or field validation has been completed.

II. Related Work and Claim Boundary

FActScore evaluates support at the level of atomic claims [5]. LEXIS produces structured security report drafts from classifier explanations [6]; agentic alert investigation uses constrained queries over logs [7]. GSAR combines typed grounding assessment and bounded recovery [8]. A recent agent-memory study finds that soft revocation labels alone can leave withdrawn information available to a generator [9]. Thus citations, structured reports, local inference, and bounded rewriting are not individually novel contributions of this manuscript.

SHAP-based cybersecurity interpretation explains influential detector features [10]; it does not by itself validate an analyst-facing factual claim. RefChecker extracts claim triplets and checks them against references [11], while SelfCheckGPT uses agreement among sampled responses to detect hallucination [12]. Our verifier instead admits exact tuples from a finite, recipient-authorized snapshot. Agreement among model samples cannot restore withdrawn permission. FaithLM evaluates whether explanations reflect model decisions through interventions [13]; our evidence-fidelity test does not establish that form of causal explanation faithfulness. These distinctions limit the novelty claim to authorization and evidence-version enforcement in a local security-report pipeline.

Our focus is a security-specific evidence-state contract: an unavailable field cannot justify a negative factual assertion, a contradiction must remain visible, and withdrawal invalidates dependent claims. This is a candidate methodological contribution whose usefulness needs comparison against existing structured reporting and typed grounding approaches. The pilot does not show superiority to these systems.

In the implemented contract, a recipient-authorized snapshot contains an incident outcome, anomaly family, coverage flag, and a list of evidence entries with identifiers, versions, activity flags, and finite catalogue values. The authorized snapshot has a digest. Corrections change a value or version; withdrawal makes an entry inactive; a new digest invalidates the old report. The present finite vocabulary does not encode arbitrary intervals or numeric units in model claims. Those are extensions, not properties established by this experiment.

A proposed claim is $c = (t, p, v, e, k)$: claim type t, predicate p, value v, optional evidence identifier e, and optional evidence version k. Types are observation, inference, unknown, and advice. Predicates in the pilot are evidence value, outcome, family, coverage, and a fixed local-log-review suggestion. A claim that names an attacker, exact count, or successful compromise is not admissible merely because a related anomaly is present; those predicates are absent from the vocabulary.

The explanation service has no authority to change the

detector score, request forbidden raw records, or execute recommendations. It consumes a frozen authorized snapshot and returns a report. An upstream detector can be wrong; the contract concerns fidelity to its evidence, not independent verification of the incident. A malicious evidence producer or a false but internally consistent record can still mislead the service.

III. Admission and Report Lifecycle

The server first constructs a recipient-authorized snapshot. A local model proposes typed claims and supporting identifiers. The verifier compares every claim tuple against a set derived deterministically from that same snapshot. This checks value equality, reference and version equality for evidence values, and the permitted claim kind for the finite predicates. Recipient scope is enforced when the snapshot is built and checked again before a report is returned. The pilot does not use a general entailment model or interval algebra.

For that vocabulary, define an admission indicator

$$A(c, E) = R(c, E) \wedge V(c, E) \wedge K(c, E) \wedge P(c, E), \quad (1)$$

where R checks evidence identifiers, V checks versions, K checks claim kinds, and P checks the finite predicate/value tuple. The authorized snapshot E is a precondition. Admission proves only that these configured checks passed. It does not certify the truth of the source evidence or any unrestricted language-model interpretation.

An admitted factual claim is rendered using a controlled template, preserving its evidence identifiers and uncertainty wording. Free-form model commentary is omitted. A citation attached to arbitrary prose is insufficient for admission. For example, a high failure-rate bin may be reported as an authorized observation, but it cannot become "the attacker tried 300 passwords."

If a proposed claim fails, the service requests at most one rewrite using the same frozen snapshot and the check failures. It does not acquire broader permissions during rewriting. If the rewrite fails or inference is unavailable, a structured fallback lists permitted observations and explicit unknowns without model-generated interpretation. This bounded recovery limits the number of calls; it does not establish hallucination freedom or a short latency bound.

The report stores the snapshot digest and the evidence identifiers and versions supporting admitted claims. A correction, authorization withdrawal, or revocation changes the authorized snapshot digest; an old report is stale and must be regenerated. An already viewed report cannot be erased from an analyst's memory. Detection decisions remain under the upstream security workflow's control.

IV. Constructed Cases and Contract Properties

Suppose authorized record E1 states that a high authentication-failure-rate bin occurred in a five-minute interval. E2 states that source identities are unavailable. E3 is a rule output labelling the event suspicious. No record establishes a successful login or identifies a threat actor. These are invented records used to illustrate the contract.

The statement "the sensor reported a high authentication-failure-rate bin during the interval" is admissible with E1. "The rule labelled the event suspicious" is admissible as an attributed inference with E3. "No external sources were involved" is not justified by E2. The correct statement is that the available evidence does not establish source identities. "A named actor compromised the account" is rejected because neither attribution nor compromise is supported.

Now suppose E1 is corrected to a medium-rate bin. A report citing the old version becomes stale. Replacing the evidence reference without updating the claim would be invalid. If E1 is revoked and its value is no longer authorized, the next report cannot repeat that value through a cached explanation. A permitted status may say that the earlier explanation is no longer supported by the current authorized snapshot, without exposing the withdrawn value.

a) Reference integrity.: Every admitted factual claim points to existing authorized evidence versions because reference resolution precedes rendering. This property assumes the validator and snapshot store operate correctly. It is not a guarantee that a cited record actually describes a real event.

b) State preservation.: The finite claim vocabulary prevents an unavailable input from licensing a negative observation and prevents an inferred record from licensing an unattributed fact. The guarantee covers controlled templates only. If unrestricted paraphrasing is allowed after admission, the paraphraser can change the meaning and must be treated as a new unverified generation.

c) Invalidation completeness.: A dependency index can identify every stored claim referencing a changed evidence version if all references are recorded. Rechecking is then mandatory before reuse. This does not guarantee that every implicit dependency was captured, nor does it retract information already disclosed. The design therefore prohibits hidden factual dependencies in admitted claims and treats unsupported free text separately.

V. Implemented Service Boundary

The explanation process is downstream of detection and incident correlation. It receives a snapshot prepared by the incident API for a particular analyst's organization and permission scope. The snapshot contains a digest, a categorical outcome, anomaly family, coverage state, and active evidence entries. Each evidence entry is a finite-catalogue value with an identifier and version. This interface deliberately excludes raw packet captures, unbounded log search, and a model-controlled query tool. A generator cannot request broader evidence by returning a new identifier. The API checks its authorization rules before assembling the snapshot; the explanation verifier checks the proposed identifiers against the assembled snapshot.

The model endpoint is an interchangeable local service rather than a hidden dependency of detection. The configured call uses JSON output, a maximum of 512 output tokens,

TABLE I
FINITE PILOT CLAIM CONTRACT AND RENDERING RULE.

Predicate	Kind	Admission condition
Evidence value	observation	Exact active evidence ID, version, and value
Outcome	inference	Exact incident rule outcome
Family	observation	Exact reported anomaly family
Coverage	observation or unknown	Exact full/partial coverage state
Review logs	advice	Fixed advisory value only

TABLE II
UNSUPPORTED CLAIM COUNTS BY SYNTHETIC CASE.

Case	Direct	Cited	No life.	Complete
Supported	1	1	0	0
Contradicted	3	0	0	0
Unresolved	0	0	0	0
Withdrawn	2	2	1	0
Corrected	0	2	1	0
Restricted	3	2	1	0
Malicious note	1	1	0	0
Model failure	0	0	0	0

a prompt-size cap, a response-size cap, one generation at a time, and a timeout. A failure does not remove the incident or change its outcome. It produces a template report from current authorized fields. When inference runs on a second laptop, the endpoint requires authenticated TLS; the local experiment used a loopback endpoint. The experiment therefore measures local generation behavior, not network transport to a second device.

Table I is intentionally small. The outcome statement is attributed to the declared correlation rule, not asserted as a confirmed compromise. Partial coverage renders as an unknown, so absent evidence does not become a claim that no threat exists. Advice is a separate kind and a fixed suggestion to review local logs; it neither queries those logs nor authorizes action. The table also exposes a limitation: an arbitrary but true security fact cannot be expressed unless the catalogue and verifier are deliberately extended. In particular, exact counts and named actors are outside the evaluated claim language.

The generator receives the snapshot as untrusted data along with the output schema. An evidence value may contain instruction-like text, but the model has no attached file, network, or shell tools. The validator does not trust the model's citation format alone: every proposed tuple must equal a permitted tuple computed from the snapshot. The renderer then produces its own sentence; it does not copy an arbitrary sentence supplied by the model. This separates two error classes. The model may emit a syntactically valid but unsupported claim, which fails membership, or it may emit invalid JSON or an invalid claim schema, which fails structural validation. Both are observable failures, not silent omissions.

The one-repair policy keeps the authorized snapshot fixed and gives the model a bounded list of check failures. It is a recovery attempt, not a second source of evidence. If either generation fails, the final output is generated from deterministic expected claims. The fallback's exact wording is less expressive than a skilled analyst's report, but its behavior is explainable and reproducible. A default fallback can be preferable when the local model is too slow or unreliable; a deployed system should not hide how often that path is taken.

VI. PILOT CASES AND SCORING

The eight cases were specified in a frozen JSON file whose digest is checked by the evaluation script before inference. They cover a supported case, a contradicted case, partial coverage, withdrawn evidence, corrected evidence, restricted evidence, instruction-like text in an evidence note, and a deliberate model failure. For a lifecycle case, each record has an initial snapshot and a current snapshot. The cited condition generates from the initial one; scoring uses the current one. This exposes stale or unauthorized claims without pretending that a prompt can foresee a later revision. The complete method regenerates from current authorized values after invalidation.

The score counts claims present in the model's structured output, and calls one unsupported when it is not admitted against the current finite contract. Unsupported does not mean that a human reviewer proved the sentence false; a claim can fail because its evidence is expired, unavailable to this recipient, or outside the implemented vocabulary. The direct and citation-prompted conditions are therefore useful stress tests of this contract, not measures of general factual accuracy. Conversely, a deterministic template scores perfectly because it is built from the same expected set. This shared construction makes the safety comparison transparent while preventing a claim of independent validation.

The no-lifecycle ablation is paired with the cited condition by reusing its raw output and applying verification against the initial snapshot. This avoids attributing a difference to a fresh random model sample. It then scores the resulting report against the current snapshot. Three claims remain unsupported: one each in withdrawn, corrected, and restricted cases (Table II). The complete path detects these changed snapshots and falls back to the current template. For unchanged cases, it may attempt one repair; every case ultimately fell back. The complete result is thus a demonstration of version enforcement coupled to a negative result about the model's ability to satisfy the contract in this configuration.

The CPU host was Windows with an AMD Ryzen 5 4500U, six logical processors, about 20.9 GB of RAM, four inference threads, and one concurrent generation. The configured context length was 8192. These settings and the 4-bit quantized Qwen3-4B variant specify the measured configuration; they do not establish that other devices, quantizations, or model sizes will behave similarly. A separate small-model memory check recorded about 2.96 GB peak process working set for Qwen3-1.7B Q4_K_M; it is not a memory measurement of the 4B pilot and is not used to claim a 4B footprint.

TABLE III
WHAT EACH COMPARISON CAN AND CANNOT ESTABLISH.

Comparison	Interpretable conclusion
Direct vs. cited	Effect of this citation instruction on finite-contract errors; model samples remain stochastic
Cited vs. verifier	Whether exact schema and tuple checks remove unsupported candidates
No lifecycle vs. complete	Whether current-snapshot validation prevents stale reuse in changed cases
Template vs. complete	Added generation cost and observed output difference; no human usefulness inference

VII. Research Questions and Comparator Semantics

The first research question asks whether finite claim admission prevents an explanation from expressing specificity absent from the recipient's current snapshot. It is evaluated by unsupported tuple counts, invalid-contract cases, and fallback rates. The second asks whether a current-version check prevents reuse of claims that were once authorized but no longer are. It is evaluated by the paired no-lifecycle ablation and three changed-snapshot cases. The third asks whether the model yields useful additional information at acceptable cost. Our data do not answer that question because both the template and complete method used the same expected-claim set and no analysts evaluated the outputs. These questions must not be collapsed into a single "explanation accuracy" number.

The template comparator constructs allowed claims without inference. It is the safety and speed reference, and its perfect finite-contract score is expected by construction. Direct generation receives the same snapshot but no explicit instruction to cite each evidence value. Citation prompting adds evidence identifiers and versions to the prompt. Neither prompt condition is allowed to query missing data. The no-lifecycle condition applies the claim checker to an initial snapshot and then tests the result against the current snapshot; this isolates the risk of assuming that one successful check stays valid indefinitely. The complete method validates the current snapshot, attempts at most one repair for unchanged inputs, and emits the deterministic fallback when no valid generation is available. The comparator definitions are explicit so a lower error count cannot be mistaken for better natural-language reasoning.

Atomic claim scoring is sensitive to the output representation. A direct answer could contain a long paragraph, but the pilot asks all model conditions for structured claims so that unsupported output can be counted with the same parser. Invalid JSON is reported separately from unsupported claims because assigning an arbitrary factual count to unparseable text would invent observations. Failure cases likewise remain visible. This protocol narrows the study to structured generation; it does not establish what the same model would do in unrestricted conversation.

The mean finite-claim coverage in the summary file uses the allowed claim set as its denominator. It is a completeness check for this schema, not evidence that the analyst received the right explanation. For example, a report can enumerate all allowed fields while giving no practical priority or useful next question. Conversely, a concise report could omit low-priority allowed fields while helping an analyst. A follow-on human evaluation should measure task accuracy, time, and uncertainty recognition. It should be preregistered and sampled beyond these eight constructed cases.

VIII. Adversary and Failure Model

Indirect prompt injection can place instructions inside externally supplied data [14]. Therefore evidence notes remain untrusted inputs even when the producer is authenticated. Local execution also differs from confidential computing: accelerator trusted execution environments protect workloads from an untrusted host [15], whereas our CPU pilot assumes a trusted host and provides no hardware attestation. An untrusted sensor note may contain ordinary data mixed with imperative text. The model may obey that text even when prompted to ignore it. The verifier's defense is narrower and testable: a note cannot introduce a new permitted claim tuple or create an authorized evidence ID. A malicious source that controls an otherwise authorized finite value can still make a false observation appear valid. Such source compromise is outside the explanation contract and must be handled by detection, provenance, and incident review. The contract prevents unauthorized *expression* from a snapshot; it does not authenticate reality.

There are three distinct stale-data paths. First, a corrected record may retain the same identifier while its version and value change. Second, a withdrawal may make a previously active record unavailable to this recipient. Third, the recipient's role or organization scope may change without a sensor correction. All three require a fresh authorization check before serving a cached report. A digest mismatch is a coarse invalidation rule: it invalidates an entire report even if a changed entry was not mentioned. That costs regeneration but avoids claiming a more precise dependency analysis than the implementation provides.

The generated output is another untrusted boundary. The parser rejects an invalid JSON shape; the checker rejects wrong kinds, predicates, values, references, or versions; the renderer maps admitted claims to fixed wording. Errors are recorded and may trigger one retry. A service timeout, malformed response, or unavailable model maps to a visible fallback reason. This fail-closed behavior depends on the verifier being outside the model and on the snapshot store enforcing recipient access. It is not obtained from the prompt's wording alone.

The dashboard is a disclosure surface. A report that is correct for one analyst may be unauthorized for another. Therefore the API checks incident scope and report scope, and the explanation cannot be treated as a shareable string detached from those scopes. Telemetry should record status, duration, and failure class without serializing prompts, credentials, or raw evidence. The present measurements do not include a full privacy attack or side-channel analysis. In particular, a derived evidence value can itself disclose sensitive information, and

TABLE IV
FROZEN EIGHT-CASE LOCAL PILOT; COUNT DENOMINATORS ARE SHOWN.

Condition	Claims	Unsupported	Fallbacks	Median s
Template	46	0	0	< 0.01
Direct	20	10	0	53.0
Cited	20	8	0	86.6
Verify only	48	3	8	–
Complete	46	0	8	146.7

a local model process can retain data in memory or logs if configured poorly.

Operational failures also affect interpretation. A slow model may leave the security decision available but delay explanatory text. A missing database or broken authorization check should prevent a current report from being served. A cached report can be useful for audit under a separate retention policy, but it cannot be silently presented as current. These outcomes are testable at the API level and should be distinguished from the model's claim quality. The pilot measures one local inference configuration; it does not measure concurrent analyst demand or second-laptop transport failure.

IX. Evaluation and Limitations

The evaluated Qwen3 model belongs to a family with distinct model sizes and thinking modes [16]. Published local–cloud offloading work optimizes quality, latency, and usage cost jointly [17]. Our experiment fixes local execution and uses deterministic fallback on failure; its measured latency does not compare cloud offloading or prove an optimal deployment choice. We froze eight synthetic cases before running the five conditions: deterministic templates, direct local generation, citation-prompted generation, verification without lifecycle checks, and the complete method. The cases exercise unavailable fields, contradictory observations, corrected or withdrawn evidence, and malicious strings embedded in evidence. The same snapshots and Qwen3-4B Q4_K_M local CPU configuration were used where generation was applicable. Direct and citation-prompted outputs were scored as atomic claims; the other conditions used the finite claim schema. The validator is an exact contract check, not a human semantic judgement.

Table IV reports aggregate counts across the eight cases, not estimates over a population of incidents. Direct generation had one failed case and two invalid-contract cases; cited generation failed in three cases. The no-lifecycle ablation retained three unsupported claims that a current-snapshot check would reject. The complete method produced no unsupported claims but used deterministic fallback in every case. The latency is the median measured generation time, not end-to-end analyst task time; the no-lifecycle condition reused cited output and has no independent timing measurement. Thus the experiment supports a narrow claim about contract enforcement while showing that this model configuration added no demonstrated benefit over templates.

The distribution in Table II matters as much as the aggregate. The contradicted case contributed three direct unsupported claims but no cited unsupported claims, while the corrected case contributed two cited unsupported claims and no direct unsupported claims. Citation prompting thus did not consistently improve each evidence condition. Withdrawal, correction, and restriction each left one unsupported claim after initial-snapshot verification. The complete method's current-snapshot check removed all three by invalidating the earlier view and using a current deterministic report. These counts come from one output per case and condition; they are not a confidence interval for future model behavior.

The deliberate model-failure case produced no model claims, so counting it as zero unsupported without also reporting the failure would reward nonproduction. Likewise, a fallback's supported claims are generated by a known finite constructor rather than by successful model reasoning. We therefore report raw claim counts, failure cases, and fallback cases together. The direct condition had 20 claims over the eight cases; the complete condition emitted 46 after fallback. A larger count in the complete output is not automatically an improvement: it reflects the template's enumeration of allowed fields, and no analyst has rated whether all of those fields were needed for a decision.

The repair attempt is deliberately bounded. For unchanged snapshots with rejected cited output, it can propose a corrected claim list from the same evidence. In this run none of those repairs led to a complete model-rendered report. The complete condition's 146.7-second median generation time includes the cited attempt and, when made, the repair; it is not the time to create the fallback text alone. It would be misleading to compare that number with the template's sub-0.01-second median as if both performed identical computation. The difference instead measures the operational cost of attempting model generation before taking the safe fallback path.

There is also an asymmetry in what these tests can falsify. A single unsupported displayed claim would violate the finite admission guarantee for that run. A run with no such claim does not prove the program correct for all possible inputs, and it certainly does not prove that the underlying incident was malicious. Integration tests exercise additional invalid identifiers, versions, and permissions, but a formal proof or exhaustive state-space test is outside this work. The pilot should be interpreted as a small, reproducible check of implementation behavior and a guide to the more demanding study that would be needed for a publication-strength usefulness claim.

The measured finite-claim coverage was 1.0 for template and complete, 0.979 for no-lifecycle, 0.221 for direct, and 0.267 for cited. This is coverage against the programmatic expected-claim set, not analyst usefulness. It rewards exhaustive template enumeration and is unsuitable as a human quality metric. The raw case records, evaluation code, machine description, and hashes are included with the repository. No independent analyst labels, field deployments, or full latency distribution are available.

The contract cannot repair inaccurate source evidence, establish missing causal links, or guarantee the soundness of arbitrary prose. Its finite predicate vocabulary restricts

expressiveness, and conservative admission can omit useful explanations. Recommendation usefulness requires a separate analyst evaluation. Prompt injection is reduced by treating evidence as data and denying model tool authority, but the generator can still produce misleading output before verification. Only the checked, controlled statements receive the stated contract guarantee.

X. Conclusion

Versioned authorized snapshots, finite claim admission, and controlled rendering make missing and withdrawn information explicit in local alert explanations. In eight frozen cases the complete method rejected unsupported outputs, yet fell back every time and had much higher latency than templates. The result establishes a reproducible safety boundary and a clear negative utility result for the tested model configuration. It does not establish source truth, human usefulness, or field reliability.

a) Acknowledgments: AI assistance.: OpenAI ChatGPT through Codex assisted with literature discovery, method formulation, drafting, and editing, including the abstract and references. The authors remain responsible for verifying every claim and complying with the selected venue's disclosure requirements. All reported measurements are from the accompanying local pilot; examples are constructed.

References

[1] M. Rigaki, C. Catania, and S. Garcia, "Hackphyr: A Local Fine-Tuned LLM Agent for Network Security Environments," arXiv:2409.11276, 2024, doi: 10.48550/arXiv.2409.11276.

[2] N. Suryanto *et al.*, "RedSage: A Cybersecurity Generalist LLM," in *Proc. ICLR*, 2026. [Online]. Available: https://openreview.net/pdf?id=W4FAenIrQ2

[3] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling Large Language Models to Generate Text with Citations," in *Proc. EMNLP*, 2023, pp. 6465–6488, doi: 10.18653/v1/2023.emnlp-main.398.

[4] L. Gao *et al.*, "RARR: Researching and Revising What Language Models Say, Using Language Models," in *Proc. ACL*, vol. 1, 2023, pp. 16477–16508, doi: 10.18653/v1/2023.acl-long.910.

[5] S. Min *et al.*, "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation," in *Proc. EMNLP*, 2023, pp. 12076–12100, doi: 10.18653/v1/2023.emnlp-main.741.

[6] M. Atoum, M. Al-Essa, Y. Alsarhan, A. K. A. Hwaitat, and M. Imran, "LLM Enhanced Explainable Intrusion Detection System for Generating Actionable Security Insights," *Computers, Materials & Continua*, vol. 89, no. 2, Art. no. 50, 2026, doi: 10.32604/cmc.2026.085403.

[7] E. Eilertsen, V. Mavroeidis, and G. Grov, "Towards Agentic Investigation of Security Alerts," in *Proc. IEEE BigData*, 2025, pp. 7793–7802, doi: 10.1109/BigData66926.2025.11402161.

[8] F. A. Kamelhar, "GSAR: Typed Grounding for Hallucination Detection and Recovery in Multi-Agent LLMs," arXiv:2604.23366, 2026, doi: 10.48550/arXiv.2604.23366.

[9] Y. T. Shen, K. Toyoda, and A. Leung, "Revoked but Still Authoritative: An Empirical Study of Revocation Enforcement in Agent-Memory Systems," arXiv:2609.08258, 2026, doi: 10.48550/arXiv.2609.08258.

[10] N. Srisumrith and S. Sodsee, "Detecting Cybersecurity Threats by Integrating Explainable AI with SHAP Interpretability and Strategic Data Sampling," arXiv:2602.19087, 2026, doi: 10.48550/arXiv.2602.19087.

[11] X. Hu *et al.*, "Knowledge-Centric Hallucination Detection," in *Proc. EMNLP*, 2024, pp. 6953–6975, doi: 10.18653/v1/2024.emnlp-main.395.

[12] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in *Proc. EMNLP*, 2023, pp. 9004–9017, doi: 10.18653/v1/2023.emnlp-main.557.

[13] Y.-N. Chuang *et al.*, "FaithLM: Towards Faithful Explanations for Large Language Models," in *Proc. EACL*, vol. 1, 2026, pp. 3802–3824, doi: 10.18653/v1/2026.eacl-long.177.

[14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. ACM AISec*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[15] K. Vaswani *et al.*, "Confidential Computing within an AI Accelerator," in *Proc. USENIX ATC*, 2023, pp. 501–518. [Online]. Available: https://www.usenix.org/conference/atc23/presentation/vaswani

[16] A. Yang *et al.*, "Qwen3 Technical Report," arXiv:2505.09388, 2025, doi: 10.48550/arXiv.2505.09388.

[17] L. Yuan, D.-J. Han, S. Wang, and C. G. Brinton, "Local-Cloud Inference Offloading for LLMs in Multi-Modal, Multi-Task, Multi-Dialogue Settings," in *Proc. ACM MobiHoc*, 2025, pp. 201–210, doi: 10.1145/3704413.3764429.